Unit #: 2

Unit Name: Counting, Sorting and Searching

Topic: Sorting

Course objectives:

The objective(s) of this course is to make students

CObj1: Acquire knowledge on how to solve relevant and logical problems using computing machine.

CObj2: Map algorithmic solutions to relevant features of C programming language constructs.

CObj3: Gain knowledge about C constructs and its associated eco-system.

CObj4: Appreciate and gain knowledge about the issues with C Standards and its respective behaviors.

CObj5: Get insights about testing and debugging C Programs.

Course outcomes:

At the end of the course, the student will be able to

CO1: Understand and apply algorithmic solutions to counting problems using appropriate C Constructs.

CO2: Understand, analyse and apply text processing and string manipulation methods using C Arrays, Pointers and functions.

CO3: Understand prioritized scheduling and implement the same using C structures.

CO4: Understand and apply sorting techniques using advanced C constructs.

CO5: Understand and evaluate portable programming techniques using pre-processor directives and conditional compilation of C Programs.

Introduction

There are so many things in our real life that we need to search for, like a particular record in the database, roll numbers in the merit list, a particular telephone number in a telephone directory, a particular page in a book etc. The concept of sorting came into existence, making it easier for everyone to arrange data in order to make the searching process easy and efficient. **Arranging the data in ascending or descending order is known as sorting.**

Why Sorting?

Think about searching for something in a sorted drawer and unsorted drawer. If the large data set is sorted based on any of the fields, then it becomes easy to search for a particular data in that set. Sometimes in real world applications, it is necessary to arrange the data in a sorted order to make the searching process easy and efficient.

Example: Candidates selection process for an Interview- Arranging candidates for the interview involves sorting process (sorting based on qualification, Experience, skills or age etc).

Sorting Algorithms

Sorting algorithm specifies the way to arrange data in a particular order. Most common orders are in numerical or lexicographical order. Depending on the data set, Sorting algorithms exhibit different time and space complexity.

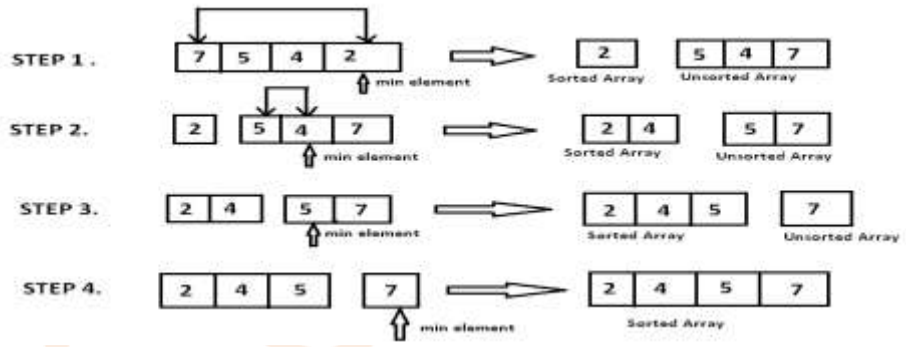
Following are samples of sorting algorithms.

1. Bubble Sort
2. Insertion Sort
3. Quick Sort
4. Merge Sort
5. Radix Sort
6. **Selection Sort -- Dealt in detail**
7. Heap Sort

Selection sort

- The method of sorting is faster as the movement of data is minimized.
- Used to arrange data in ascending order, the smallest number is selected and placed at the beginning of the collection.
- When the next pass takes place the second minimum number is selected and placed in the next position.
- The process is repeated until all the elements are placed in the correct order.
- With each pass the unsorted elements are reduced by 1 as one element is sorted and placed in its final position.

Pictorial Representation



Selection Sort Algorithm

The selection sort algorithm sorts an array by repeatedly finding the minimum element (considering ascending order) from the unsorted part and putting it at the beginning. It is **an in-place comparison** sorting algorithm. The algorithm **maintains two sub-arrays in a given array.**

1) The sub-array which is already sorted.

2) Remaining sub-array which is unsorted.

Initially, the sorted part of the array is empty and unsorted part is the given array. Sorted part is placed at the left, while the unsorted part is placed at the right.

For sorting in ascending order, in every iteration, the minimum element from the unsorted sub-array is picked and moved to the sorted subarray.

For sorting in descending order, in every iteration, the maximum element from the unsorted sub-array is picked and moved to the sorted sub-array.

Coding Example_1: Selection sort on array of integers

```
#include<stdio.h>
void sort(int *a, int n);
int main()
{
	int a[] = {22,19,14,25,99,81};
	int n = sizeof(a)/sizeof(*a);

	printf("before sorting\n");
	for(int i = 0; i < n; i++)
	{
		printf("%d\t",a[i]);
	}

	sort(a,n);

	printf("\nafter sorting\n");
	for(int i = 0; i < n; i++)
	{
		printf("%d\t",a[i]);
	}

	return 0;
}
```

```
void sort(int *a, int n)
{
        int pos;
        for(int i = 0; i< n-1;i++)
        {
                pos = i;
                for(int j = i+1; j<=n-1; j++)   // or j < n
                {
                        if(a[pos] > a[j])
                        {
                                pos = j;
                        }
                }
                if(pos != i)
                {
                        int t = a[pos];
                        a[pos] = a[i];
                        a[i] = t;
                        //swap(&a[pos], &a[i]);
                }
        }
}
```

Coding Exampe_2: Sort the elements of the array given by the user

```
#include<stdio.h>
void read(int a[],int n);
void disp(int a[],int n);
void sort(int a[],int n);
void swap(int *x,int *y);
```

```
int main()
{
        int a[100];
        int n;
        printf("enter the number of elements u want to sort\n");
        scanf("%d",&n);

        printf("enter %d elements\n",n);
        read(a,n);

        printf("before sorting\n");
        disp(a,n);

        sort(a,n);

        printf("after sorting\n");
        disp(a,n);

        return 0;
}

void read(int a[],int n)
{
        for(int i = 0;i<n;i++)
        {
                scanf("%d",&a[i]);
        }
}
```

```
void disp(int a[],int n)
{
        for(int i = 0;i<n;i++)
        {
                printf("%d ",a[i]);
        }
        printf("\n");
}

void swap(int *x,int *y)
{
        int temp = *x;
        *x = *y;
        *y = temp;
}
void sort(int a[],int n)
{
        int i,pos,j;
        for(i = 0;i<n-1;i++)
        {
                pos = i;
                for(j = i+1;j<n;j++)
                {
                        if(a[pos]>a[j])
                                pos = j;
                }
                if(pos != i)     // think about this – Is it really required?
                        swap(&a[pos],&a[i]);
        }
}
```